

Project Title

Unicode Pattern Search Engine for Indian Language Wikipedia

Abstract:

The Unicode Pattern Search Engine is designed to provide fast and efficient keyword searching across large Indian-language Wikipedia datasets. The system addresses the limitations of traditional substring searching, which can become inefficient when handling large multilingual corpora and complex Unicode scripts such as Telugu, Devanagari, Tamil, and Bengali. It uses Unicode-aware text preprocessing to handle multilingual content effectively.

The proposed system implements advanced Data Structures and Algorithms, including KMP, Rabin-Karp, and Aho-Corasick for efficient pattern matching. An inverted index is used to reduce search time and support large datasets, while Hash Tables assist in fast lookup and frequency-based processing. The system supports single and multi-keyword queries, ranks results based on relevance, and provides contextual snippets to help users quickly identify useful information.

Description:

Unicode Search Engine is a multilingual text-search system designed to efficiently search content written in **English, Telugu, Hindi, Tamil, and Bengali**. The project focuses on applying **Data Structures and Algorithms** to solve the challenges of searching large Unicode-based text collections, particularly Indian-language Wikipedia articles.

The system uses an **Inverted Index** to quickly identify candidate documents instead of unnecessarily scanning the entire corpus. After candidate retrieval, handwritten pattern-matching algorithms such as **Knuth-Morris-Pratt (KMP), Rabin-Karp, and Aho-Corasick** are used to find occurrences of the searched keywords. Unicode normalization is performed so that text written using different Unicode representations can be processed consistently.

The search engine supports both **single-keyword and multi-keyword searches**. Single-keyword searches can use KMP or Rabin-Karp, while multi-keyword searches use the **Aho-Corasick algorithm**, allowing up to **50 keywords in a single query**. The system also calculates match frequency and relevance scores and displays **contextual snippets** from matching documents.

Code:

AhoCorasick Algorithm:

```
package com.unicodesearch.algorithm;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Queue;

public class AhoCorasick extends Trie {

    private boolean built = false;

    public AhoCorasick(List<String> keywords) {
        for (String kw : keywords) {
            if (kw != null && !kw.isEmpty()) insert(kw);
        }
        build();
    }

    /** BFS construction of failure links and output propagation, run once after all
    private void build() {
        Queue<TrieNode> queue = new ArrayDeque<>();
        for (TrieNode child : root.children.values()) {
            child.failureLink = root;
            queue.add(child);
        }

        while (!queue.isEmpty()) {
            TrieNode current = queue.poll();

            for (Map.Entry<Character, TrieNode> entry : current.children.entrySet()) {
                char c = entry.getKey();
                TrieNode child = entry.getValue();

                TrieNode fallback = current.failureLink;
                while (fallback != null && fallback.getChild(c) == null) {
                    fallback = fallback.failureLink;
                }
                child.failureLink = (fallback == null) ? root : fallback.getChild(c);
                if (child.failureLink == child) child.failureLink = root;

                child.outputs = new ArrayList<>(child.failureLink.outputs);
                if (child.isEndOfWord) child.outputs.add(index: 0, child.word);

                queue.add(child);
            }
        }
        built = true;
    }

    public Map<String, List<Integer>> searchAll(String text) {
        Map<String, List<Integer>> matches = new HashMap<>();
        TrieNode current = root;

        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);

            while (current != root && current.getChild(c) == null) {
                current = current.failureLink;
            }
            TrieNode next = current.getChild(c);
            current = (next != null) ? next : root;

            for (String matchedWord : current.outputs) {
                int startPos = i - matchedWord.length() + 1;
```

```
                int startPos = i - matchedWord.length() + 1;
```

```

        for (String matchedWord : current.outputs) {
            int startPos = i - matchedWord.length() + 1;
            matches.computeIfAbsent(matchedWord, k -> new ArrayList<>()).add(s);
        }
    }
    return matches;
}

public boolean isBuilt() {
    return built;
}
}

```

KMP Algorithm:

```

package com.unicodesearch.algorithm;

import com.unicodesearch.model.MatchResult;
import java.util.ArrayList;
import java.util.List;

public class KMP {

    /** Builds the Longest Prefix Suffix (failure function) array for the pattern. */
    private static int[] buildLPS(String pattern) {
        int m = pattern.length();
        int[] lps = new int[m];
        int len = 0; // length of the previous longest prefix-suffix
        int i = 1;
        lps[0] = 0;
        while (i < m) {
            if (pattern.charAt(i) == pattern.charAt(len)) {
                len++;
                lps[i] = len;
                i++;
            } else if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
        return lps;
    }
}

```

```

public static MatchResult search(String text, String pattern) {
    long start = System.nanoTime();
    List<Integer> positions = new ArrayList<>();
    int n = text.length();
    int m = pattern.length();

    if (m == 0 || m > n) {
        return new MatchResult(pattern, positions, elapsedMs(start));
    }

    int[] lps = buildLPS(pattern);
    int i = 0; // index into text
    int j = 0; // index into pattern

    while (i < n) {
        if (text.charAt(i) == pattern.charAt(j)) {
            i++;
            j++;
            if (j == m) {
                positions.add(i - j);
                j = lps[j - 1]; // continue searching for further occurrences
            }
        } else if (j != 0) {
            j = lps[j - 1]; // use failure function instead of restarting
        } else {
            i++;
        }
    }

    return new MatchResult(pattern, positions, elapsedMs(start));
}

private static double elapsedMs(long startNanos) {
    return (System.nanoTime() - startNanos) / 1_000_000.0;
}

```

Rabin karp Algorithm:

```

ode-pattern-search-engine > backend > src > main > java > com > unicodesearch > algorithm > RabinKarp
package com.unicodesearch.algorithm;

import com.unicodesearch.model.MatchResult;

import java.util.ArrayList;
import java.util.List;

public class RabinKarp {

    private static final long PRIME = 1_000_000_007L;
    private static final long BASE = 256L;

    /**
     * Searches for all occurrences of pattern in text.
     *
     * @param text text to search
     * @param pattern pattern to find
     * @return MatchResult containing all matching positions
     */
    public static MatchResult search(String text, String pattern) {

        long start = System.nanoTime();

        List<Integer> positions = new ArrayList<>();

        if (text == null || pattern == null) {
            return new MatchResult(
                pattern == null ? "" : pattern,
                positions,
                elapsedMs(start)
            );
        }
    }
}

```

```

int n = text.length();
int m = pattern.length();

if (m == 0 || m > n) {
    return new MatchResult(
        pattern,
        positions,
        elapsedMs(start)
    );
}

long highOrder = 1;

for (int i = 0; i < m - 1; i++) {
    highOrder = (highOrder * BASE) % PRIME;
}

long patternHash = 0;
long windowHash = 0;

for (int i = 0; i < m; i++) {
    patternHash =
        (BASE * patternHash + pattern.charAt(i))
        % PRIME;

    windowHash =
        (BASE * windowHash + text.charAt(i))
        % PRIME;
}

```

```

if (patternHash == windowHash) {
    if (text.regionMatches(i, pattern, offset: 0, m)) {
        positions.add(i);
    }
}

if (i < n - m) {
    windowHash =
        (BASE
         * (windowHash
            - text.charAt(i) * highOrder)
         + text.charAt(i + m))
        % PRIME;

    if (windowHash < 0) {
        windowHash += PRIME;
    }
}

return new MatchResult(
    pattern,
    positions,
    elapsedMs(start)
);

```

```
        return new MatchResult(  
            pattern,  
            positions,  
            elapsedMs(start)  
        );  
    }  
  
    private static double elapsedMs(long startNanos) {  
        return (System.nanoTime() - startNanos)  
            / 1_000_000.0;  
    }  
}
```

CorpusLoader.java:

```

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;

public class CorpusLoader {

    public List<Document> loadCorpus(Path corpusRoot) throws IOException {
        List<Document> documents = new ArrayList<>();
        if (!Files.isDirectory(corpusRoot)) {
            throw new IOException("Corpus directory not found: " + corpusRoot.toAbsolutePath());
        }

        try (Stream<Path> languageDirs = Files.list(corpusRoot)) {
            for (Path langDir : languageDirs.filter(Files::isDirectory).sorted().toList()) {
                String language = langDir.getFileName().toString();
                try (Stream<Path> files = Files.list(langDir)) {
                    for (Path file : files.filter(p -> p.toString().endsWith(suffix: ".txt")).sorted().toList()) {
                        String fileRaw = Files.readString(file, StandardCharsets.UTF_8);
                        String id = language + "-" + file.getFileName().toString().replace(target: ".txt", replacement: "");

                        String title;
                        String raw;
                        if (fileRaw.startsWith(prefix: "TITLE:")) {
                            int newline = fileRaw.indexOf(ch: '\n');
                            title = fileRaw.substring("TITLE:".length(), newline < 0 ? fileRaw.length() : newline).trim();
                            raw = (newline < 0) ? "" : fileRaw.substring(newline + 1).stripLeading();
                        } else {
                            title = deriveTitle(language, file.getFileName().toString());
                            raw = fileRaw;
                        }

                        String normalized = UnicodeUtil.preprocess(raw);
                        documents.add(new Document(id, title, language, normalized, raw));
                    }
                }
            }
        }

        return documents;
    }

    private String deriveTitle(String language, String filename) {
        String base = filename.replace(target: ".txt", replacement: "").replace(target: "document", replacement: "Document ");
        String cap = language.substring(beginIndex: 0, endIndex: 1).toUpperCase() + language.substring(beginIndex: 1);
        return cap + " " + base.trim();
    }
}

```

```

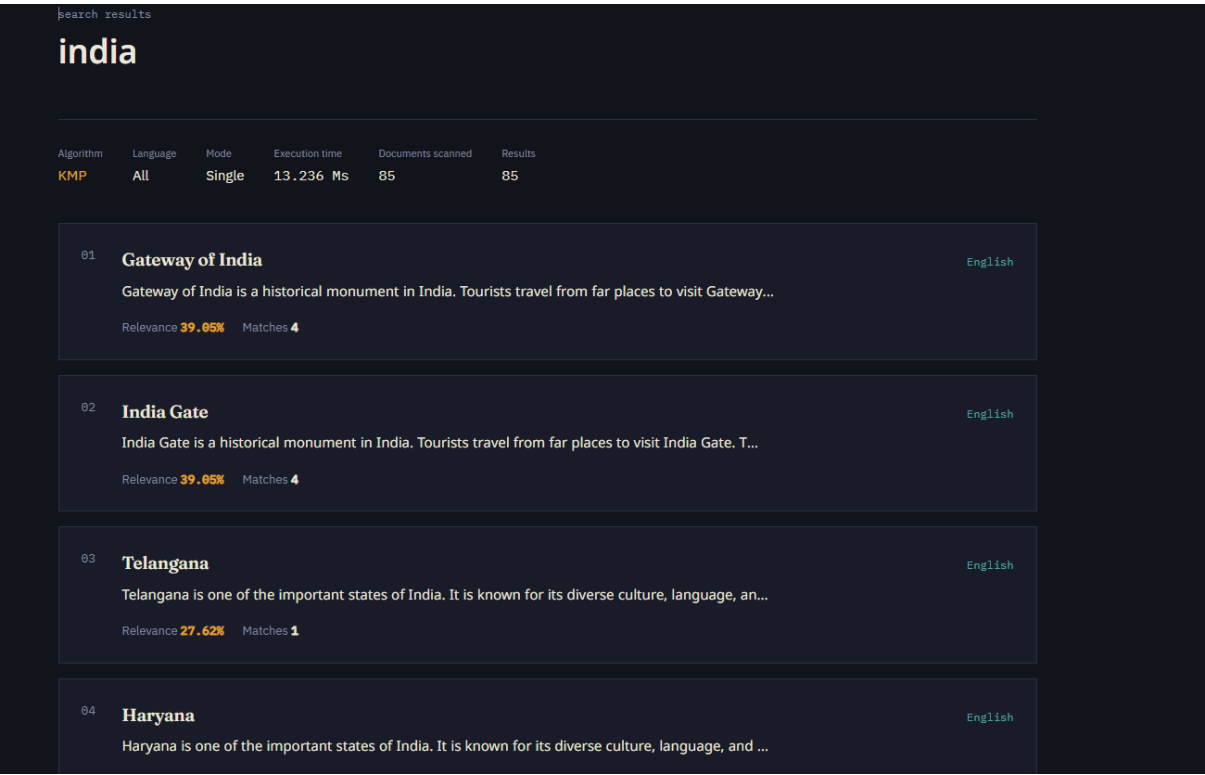
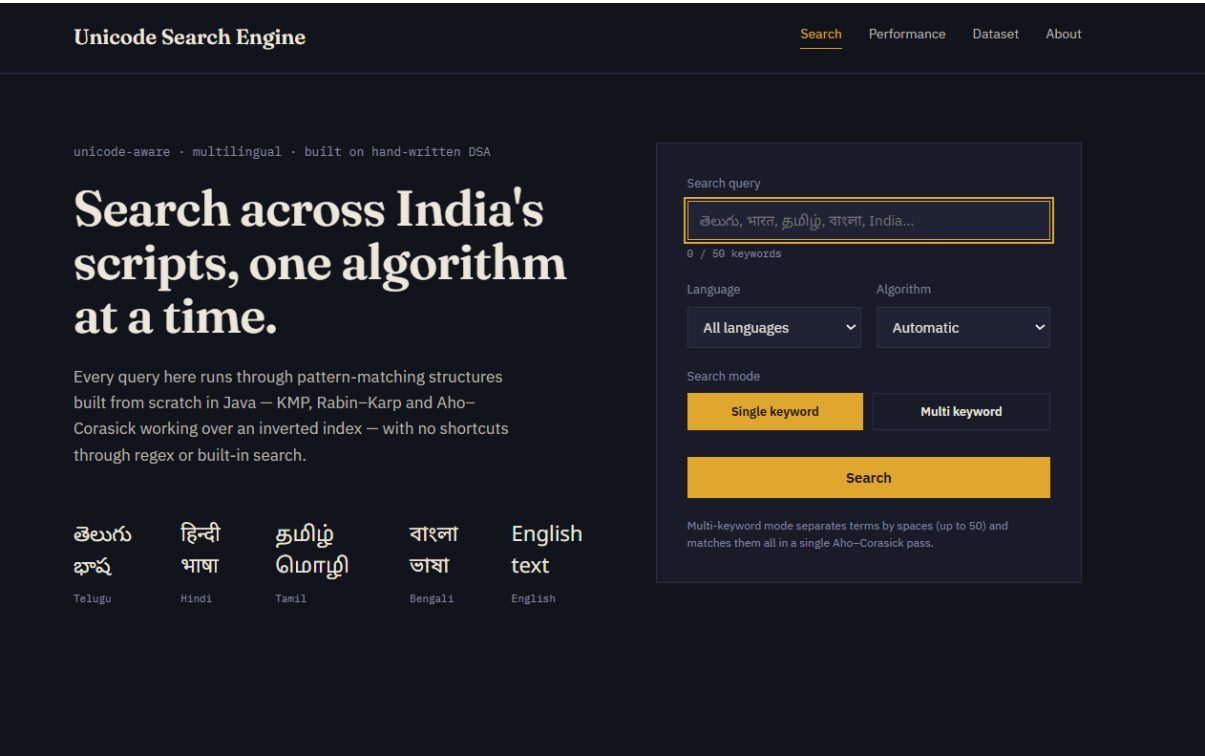
        String normalized = UnicodeUtil.preprocess(raw);
        documents.add(new Document(id, title, language, normalized, raw));
    }
}

return documents;
}

private String deriveTitle(String language, String filename) {
    String base = filename.replace(target: ".txt", replacement: "").replace(target: "document", replacement: "Document ");
    String cap = language.substring(beginIndex: 0, endIndex: 1).toUpperCase() + language.substring(beginIndex: 1);
    return cap + " " + base.trim();
}
}

```

Outputs:



04

Haryana

English

Haryana is one of the important states of India. It is known for its diverse culture, language, and ...

Relevance **27.62%** Matches **1**

05

Brihadeeswarar Temple

English

Brihadeeswarar Temple is a historical monument in India. Tourists travel from far places to visit Br...

Relevance **27.62%** Matches **1**

06

Uttar Pradesh

English

Uttar Pradesh is one of the important states of India. It is known for its diverse culture, language...

Relevance **27.62%** Matches **1**

07

Bihar

English

Bihar is one of the important states of India. It is known for its diverse culture, language, and tr...

Relevance **27.62%** Matches **1**

08

West Bengal

English

West Bengal is one of the important states of India. It is known for its diverse culture, language, ...

Relevance **27.62%** Matches **1**